

6b) MAP to implement single link list to simulate stack & Queue operations.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *top = NULL;
struct Node *front = NULL;
struct Node *rear = NULL;

struct Node *createNode(int value) {
    struct Node *newNode = (struct Node *) malloc (size of
                                                (struct Node));

    if (!newNode) {
        printf ("Memory allocation failed!\n");
        exit(0);
    }

    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}

void push (int value) {
    struct Node *newNode = createNode (value);
    newNode->next = top;
    top = newNode;
    printf ("!d pushed onto the stack.\n", value);
}
```



```

void pop() {
    if (top == NULL) {
        printf("Stack is empty. Nothing to pop.\n");
        return;
    }
    struct Node *temp = top;
    printf("%d popped from the stack.\n", top->data);
    top = top->next;
    free(temp);
}

```

```

void displayStack() {
    struct Node *temp = top;
    if (temp == NULL) {
        printf("Stack is empty.\n");
        return;
    }
    printf("Stack (Top to Bottom): ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

```

```

void enqueue(int value) {
    struct Node *newNode = createNode(value);
    if (rear == NULL) {
        front = rear = newNode;
    }
    else {
        rear->next = newNode;
    }
}

```



```

    rear = newNode;
}
printf("%d enqueued to the queue.\n", value);

```

```

void dequeue() {
    if (front == NULL) {
        printf("Queue is empty. Nothing to dequeue.\n");
        return;
    }
    struct Node *temp = front;
    printf("%d dequeued from the queue.\n", front->data);
    front = front->next;
    if (front == NULL) {
        rear = NULL;
    }
    free(temp);
}

```

```

void displayQueue() {
    struct Node *temp = front;
    if (temp == NULL) {
        printf("Queue is empty.\n");
        return;
    }
    printf("Queue (front to Rear): ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

```



```

int main() {
    int choice, value, ch;
    while (1) {
        printf("\n --- Singly Linked list Simulation --- \n");
        printf("1. Stack Operations \n");
        printf("2. Queue Operations \n");
        printf("3. Exit \n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                while (1) {
                    printf("\n --- Stack Menu --- \n");
                    printf("1. Stack Operations \n");
                    printf("2. Queue Operations \n");
                    printf("3. Exit \n");
                    printf("Enter your choice: ");
                    scanf("%d", &ch);

                    switch (ch) {
                        case 1:
                            printf("1. Push \n");
                            printf("2. Pop \n");
                            printf("3. Display Stack \n");
                            printf("4. Back to Main Menu \n");
                            printf("Enter your choice: ");
                            scanf("%d", &ch);

                            switch (ch) {
                                case 1:
                                    printf("Enter value to push: ");
                                    scanf("%d", &value);
                                    push(value);
                                    break;
                                case 2:
                                    pop();
                                    break;
                                case 3:
                                    display();
                                    break;
                                case 4:
                                    break;
                            }
                        case 2:
                            break;
                        case 3:
                            break;
                    }
                }
            case 2:
                break;
            case 3:
                break;
        }
    }
}

```



```

        break;
    case 3:
        displayStack();
        break;
    case 4:
        goto mainMenu;
    default:
        printf("Invalid choice.\n");
    }
}
break;

```

case 2:

```

while (1) {
    printf("\n --- Queue Menu --- \n");
    printf("1. Enqueue\n");
    printf("2. Dequeue\n");
    printf("3. Display Queue\n");
    printf("4. Back to Main Menu\n");
    printf("Enter your choice: ");
    scanf("%d", &ch);

```

```

    switch (ch) {

```

```

        case 1:

```

```

            printf("Enter value to enqueue: ");

```

```

            scanf("%d", &value);

```

```

            enqueue(value);

```

```

            break;

```

```

        case 2:

```

```

            dequeue();

```

```

            break;

```

```

        case 3:

```

```

            displayQueue();

```

```

            break;

```


case 4 :

goto main-menu;

default:

printf("Invalid choice. \n");

}

}

break;

case 3 :

printf("Exiting program.\n");

exit(0);

default:

printf("Invalid choice. Try again. \n");

}

main-menu: ;

}

return 0;

}

